Express.js Notes (Developer Level)

Express is the most popular backend framework for Node.js. It sits on top of Node's built-in HTTP module and makes creating APIs and servers much easier. It is the backbone of many MERN applications. [Wikipedia+1]

---

1. What is Express?

Without Express:

```
JavaScriptconsthttp=require("http");

constserver=http.createServer((req, res) => {
res.end("Hello");
});

server.listen(3000);
```

With Express:

```
JavaScriptconstexpress=require("express");
constapp=express();

app.get("/", (req, res) => {
res.send("Hello");
});

app.listen(3000);
```

Benefits:

- Easier routing
- Middleware support
- Request parsing
- API creation
- Error handling

- Template engines (EJS)

---

2. Installing Express

Bashnpm init -y
npm install express

Check package.json:

JSON{
  "dependencies": {
    "express": "^5.x.x"
  }
}

---

3. Basic Express Server

JavaScriptconstexpress=require("express");
constapp=express();

app.listen(3000, () => {
console.log("Server running");
});

---

4. First Route

JavaScriptapp.get("/", (req, res) => {
res.send("Welcome");
});

Visit:

localhost:3000/

Response:

Welcome

A route is:

```
app.METHOD(PATH, HANDLER)
```

Example:

```
app.get("/", handler)
```

Where:

- METHOD → GET, POST, PUT, DELETE
- PATH → URL
- HANDLER → callback function [GeeksforGeeks+1]

---

5. Understanding req and res

```
app.get("/", (req, res) => {
res.send("Hello");
});
```

req

Contains information sent by client.

```
req.params
req.query
req.body
req.headers
req.method
req.url
```

res

Used to send response.

```
res.send()
res.json()
res.status()
res.render()
res.redirect()
```

Express automatically provides req and res to route handlers. [Reddit+1]

---

6. Sending Responses

res.send()

JavaScriptres.send("Hello");

JavaScriptres.send("<h1>Hello</h1>");

JavaScriptres.send([1,2,3]);

---

res.json()

Most common in APIs.

```
JavaScriptres.json({
   name: "John",
   age: 22
});
```

Output:

```
JSON{
  "name":"John",
  "age":22
}
```

---

res.status()

JavaScriptres.status(404).send("Not Found");

```
JavaScriptres.status(500).json({
   error: "Server Error"
});
```

---

res.redirect()

JavaScriptres.redirect("/home");

---

res.sendFile()

JavaScriptres.sendFile(__dirname+"/index.html");

---

7. HTTP Methods

GET

Retrieve data.

```
JavaScriptapp.get("/users", (req,res)=>{
res.send("All users");
});
```

POST

Create data.

```
JavaScriptapp.post("/users",(req,res)=>{
res.send("User Created");
});
```

PUT

Update complete resource.

```
JavaScriptapp.put("/users/:id",(req,res)=>{
res.send("Updated");
});
```

PATCH

Partial update.

```
JavaScriptapp.patch("/users/:id",(req,res)=>{
res.send("Partially Updated");
});
```

DELETE

Delete resource.

```
JavaScriptapp.delete("/users/:id",(req,res)=>{
res.send("Deleted");
});
```

---

8. Routing

Routing decides what happens when a URL is hit. [Express.js+1]

```
JavaScriptapp.get("/", ()=>{});

app.get("/about", ()=>{});

app.post("/login", ()=>{});
```

---

9. Route Parameters (Path Parameters)

Used when identifying a specific resource.

```
JavaScriptapp.get("/users/:id",(req,res)=>{
res.send(req.params.id);
});
```

Request:

/users/25

Output:

25

Multiple Parameters

JavaScriptapp.get("/users/:userId/posts/:postId",(req,res)=>{
console.log(req.params);
});

URL:

/users/10/posts/99

Output:

JavaScript{
userId: "10",
postId: "99"
}

[Express.js+1]

---

10. Query Strings

Used for filtering/searching.

URL:

/products?category=mobile&page=2

Route:

JavaScriptapp.get("/products",(req,res)=>{
console.log(req.query);
});

Output:

JavaScript{
category: "mobile",
page: "2"
}

Query strings are available through req.query. [GeeksforGeeks+2Chanh Le+2]

Path Params vs Query Params

Path Param

/users/10

JavaScriptreq.params.id

Represents a specific resource.

Query Param

/users?page=2

JavaScriptreq.query.page

Used for filtering, sorting, pagination. [Reddit]

---

11. Middleware

Middleware runs before route handlers.

```
JavaScriptapp.use((req,res,next)=>{
console.log("Middleware");
next();
});
```

Flow:

```
Request
 ↓
Middleware
 ↓
Route
 ↓
Response
```

If you forget:

JavaScriptnext();

Request hangs forever.

---

12. Built-in Middleware

Parse JSON

JavaScriptapp.use(express.json());

Now:

JavaScriptreq.body

works.

Example:

```
JavaScriptapp.post("/user",(req,res)=>{
console.log(req.body);
});
```

URL Encoded Forms

JavaScriptapp.use(express.urlencoded({extended:true}));

Used for HTML forms.

---

13. Request Body

Client sends:

```
JSON{
  "name":"John"
}
```

Server:

```
JavaScriptapp.post("/user",(req,res)=>{
console.log(req.body.name);
});
```

Need:

JavaScriptapp.use(express.json());

---

14. Route Chaining

Instead of:

JavaScriptapp.get("/users",handler);

app.post("/users",handler);

Use:

JavaScriptapp.route("/users")
.get((req,res)=>{})
.post((req,res)=>{})
.delete((req,res)=>{});

[GeeksforGeeks]

---

15. Express Router

For large projects.

userRoutes.js

JavaScriptconstrouter=express.Router();

router.get("/", ()=>{
});

router.post("/", ()=>{
});

module.exports =router;

app.js

JavaScriptconstuserRoutes=require("./routes/userRoutes");

app.use("/users", userRoutes);

Routes become:

/users/
/users/123

[Express.js]

---

16. 404 Route

Must be last.

```
JavaScriptapp.use((req,res)=>{
res.status(404).send("Page Not Found");
});
```

[Express.js]

---

17. Error Handling Middleware

```
JavaScriptapp.use((err, req, res, next)=>{
res.status(500).json({
      message: err.message
  });
});
```

---

18. Nodemon

Automatically restarts server.

Install:

```
Bashnpm install -D nodemon
```

or globally:

```
Bashnpm install -g nodemon
```

Package.json:

JSON"scripts": {
  "dev": "nodemon app.js"
}

Run:

Bashnpm run dev

---

19. Environment Variables

Install:

Bashnpm install dotenv

.env

envPORT=5000

app.js

JavaScriptrequire("dotenv").config();

app.listen(process.env.PORT);

---

20. Serving Static Files

Folder:

public/
  style.css
  image.png

JavaScriptapp.use(express.static("public"));

Now:

localhost:3000/style.css

works.

---

21. Express Project Structure (MERN Standard)

```
project
│
├── node_modules
├── routes
│   └── userRoutes.js
├── controllers
│   └── userController.js
├── models
│   └── User.js
├── middleware
│   └── auth.js
├── public
├── .env
├── app.js
└── package.json
```

Things Every MERN Backend Developer Must Know

✅ Express installation
✅ Routes & HTTP methods
✅ req, res objects
✅ Path parameters (req.params)
✅ Query strings (req.query)
✅ Request body (req.body)
✅ Middleware (app.use)
✅ Express Router
✅ JSON responses
✅ Status codes
✅ Error handling
✅ 404 handling
✅ Nodemon
✅ dotenv
✅ Static files
✅ Basic REST API structure

After mastering these, your next Express topics should be:

1. Authentication (JWT)
2. Cookies
3. Sessions
4. MongoDB + Mongoose
5. MVC Architecture
6. File Uploads (Multer)
7. CORS
8. Async Error Handling
9. REST API Best Practices
10. Express Security (Helmet, Rate Limiting)